Version corrigée du 2025-03-16

À rendre le 24/03/2025

DM Informatique

Consignes :

- Il est conseillé de passer environ **6 h** sur un DM d'informatique.
- Vous préciserez obligatoirement les **spécifications** de toutes les fonctions que vous définirez en python. Ces fonctions seront de préférence commentées. Les spécifications et les commentaires seront d'une autre **couleur**.
- Vous respecterez l'**indentation** des programmes python.

I – Classification d'iris

Les fleurs iris sont des plantes à fleurs très appréciées pour leur beauté et leur diversité. Elles appartiennent à la famille des iridacées et sont caractérisées par leurs pétales colorés et leurs formes élégantes. Les trois variétés d'iris les plus connues sont l'iris setosa, l'iris versicolor et l'iris virginica. Ces trois variétés présentent des différences subtiles dans la forme et la taille de leurs pétales et de leurs sépales, comme illustré sur la Fig. 1. Des mesures de la largeur ℓ et de la longueur L des pétales ont été effectuées sur un ensemble de $N = 150$ iris, représentée par une liste de points du plan appelée `iris`. Ces mesures sont représentées dans le plan par les points (ℓ, L) sur la Fig. 1.

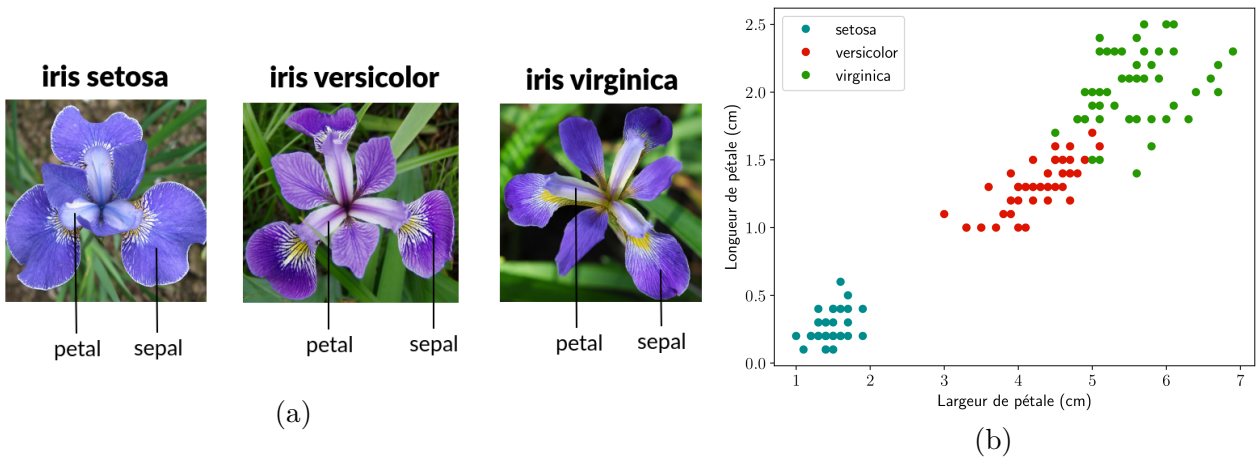


Fig. I.1 – (a) Trois variétés d'iris : setosa, versicolor et virginica. (b) Représentation des données iris dans le plan (ℓ, L), où ℓ est la largeur et L la longueur des pétales.

1) Rappeler la différence entre un apprentissage supervisé et un apprentissage non supervisé. Quel algorithme de classement standard est associé à chaque type d'apprentissage ?

Solution

> *Voir cours : apprentissage supervisé = données étiquetées (algorithme des k plus proches voisins), apprentissage non supervisé = données non étiquetées (algorithme des k-moyennes).*

I.1 – Algorithme des k plus proches voisins

2) Définir mathématiquement la distance euclidienne $d(y_1, y_2)$ entre deux points $y_1 = (\ell_1, L_1)$ et $y_2 = (\ell_2, L_2)$ du plan. La définir en `python` en complétant le code ci-dessous et en utilisant la fonction `np.sqrt` de la bibliothèque `numpy`. Quelle autre distance connaissez-vous ?

```
def distance(y1,y2):
  """
  Calcule la distance euclidienne entre deux paires de coordonnées y1 et y2.
  """
  return ...
```

Solution

Mathématiquement,

$$\boxed{d(y_1, y_2) = \sqrt{(\ell_1 - \ell_2)^2 + (L_1 - L_2)^2}.}$$

En python,

```
def d(y1,y2):
    return ((y1[0]-y2[0])**2 + (y1[1]-y2[1])**2)**0.5
```

3) On dispose de la classification `varietes` (de type dictionnaire) qui associe à chaque iris sa variété. Les classes correspondent aux chaînes de caractère `'setosa'`, `'versicolor'` et `'virginica'`. Que renvoie `varietes[(6,1.8)]` en supposant que `(6,1.8)` est bien un élément de la liste `iris` ?

Solution

L'appel `varietes[(6,1.8)]` *renvoie la variété* `'virginica'`.

4) Selon vous, où devrait être classé un nouvel iris de coordonnées $(\ell = 2.5, L = 0.5)$? de coordonnées $(\ell = 3.5, L = 1.7)$? Justifier.

Solution

Le point $(2.5, 0.5)$ *est proche des iris de la variété setosa, tandis que le point* $(3.5, 1.7)$ *est proche des iris de la variété versicolor. Il est donc probable que le premier iris soit classé dans la variété* `'setosa'` *et le second dans la variété* `'versicolor'`.

5) On souhaite appliquer l'algorithme des k plus proches voisins à l'ensemble d'apprentissage `iris` et à la classification `varietes` pour classer un nouvel iris de coordonnées $(\ell = 2.5, L = 0.5)$. On commence pour cela par définir la fonction `voisins` suivante. Qu'effectue cette fonction ? Effectuer l'appel pas à pas de cette fonction sur l'ensemble `E = [(0,0),(1,1),(2,2),(3,3),(4,4)]`, `x=(1.9,1.9)`, `k=3` et `d=distance`.

```
def voisins(x, E, k, d):
    indices = sorted(E, key=lambda y: d(x, y))
    return indices[:k]
```

Solution

La fonction `voisins` *renvoie les* k *points les plus proches voisins de la donnée* x *dans l'ensemble* E*. La liste indices contient les points de* E *triés par ordre croissant de leur distance à* x*.*

6) On définit maintenant la fonction `classemaj` que renvoie la classe majoritaire de la liste des k plus proches voisins d'une nouvelle donnée x. Préciser les spécifications de cette fonction (variables d'entrée, variables de sortie). Effectuer l'exécution pas à pas de l'appel de cette fonction avec les arguments `E = [(0,0),(1,1),(2,2),(3,3),(4,4)]`, `x=(1.9,1.9)`, `k=3`, `d=distance`, et `p={(0,0):'a',(1,1):'b',(2,2):'b',(4,4):'b'}`. Quelle classe majoritaire est prédite sur cet exemple simple ?

```
def classemaj(E, x, d, k, p):
  v = voisins(E, x, d, k)            # k plus proches voisins de x
  c = {p[y]:0 for y in v}            # liste des classes des voisins
  for y in v:                        # boucle sur voisins
      c[p[y]] += 1                   # comptage des classes
  maxi = 0                           # initialisation maximum
  for i in c:                        # boucle sur classes des voisins
      if c[i] > maxi:                # si classe i plus nombreuse
          maxi, classe = c[i], i     # mise à jour classe majoritaire
  return classe                      # retourne classe majoritaire
```

Solution

L'appel de la fonction `classemaj` *avec les arguments donnés renvoie la classe majoritaire* `'b'`.

7) En appliquant la fonction `classemaj` à l'ensemble d'apprentissage `E=iris` et à la classification `p=varietes`, sur la nouvelle donnée ($\ell = 2.5, L = 0.8$), on obtient les résultats suivants suivant la valeur de k : 'setosa' pour $k = 1$, 'versicolor' pour $k = 3$, 'setosa' pour $k = 5$, 'setosa' pour $k = 7$. Commenter ces résultats. Pourquoi seules les valeurs de k impaires sont-elles utilisées ?

Solution

Pour $k = 1$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche d'un seul iris de cette variété. Pour $k = 3$, la classe majoritaire est 'versicolor' car le point $(2.5, 0.8)$ est proche de deux iris de cette variété et d'un iris de la variété 'setosa'. Pour $k = 5$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche de trois iris de cette variété et de deux iris de la variété 'versicolor'. Pour $k = 7$, la classe majoritaire est 'setosa' car le point $(2.5, 0.8)$ est proche de quatre iris de cette variété et de trois iris de la variété 'versicolor'. La classe prédite dépend du nombre de voisins ! Les valeurs de k impaires sont utilisées pour éviter les cas d' $\boxed{\text{égalitédevoisinage}}$.

I.2 – Qualité des prédictions

Les données de l'ensemble `iris` précédent sont déjà classéEs dans le dictionnaire `varietes` et peuvent servir d'ensemble test. On souhaite quantifier la qualité du classement par l'algorithme des k plus proches voisins. On définit alors la matrice de confusion M associée à cet ensemble test, telle que M_{ij} est le nombre d'iris de la classe réelle i dont la classe attribuée par l'algorithme des k plus proches voisins est j. Sur les $N = 150$ iris, pour une certaine valeur de k, on obtient la matrice de confusion de la Tab. I.1.

	setosa	versicolor	virginica
setosa	46	3	0
versicolor	1	48	2
virginica	0	5	45

Table I.1 – Matrice de confusion.

8) La précision P de l'algorithme des k plus proches voisins est définie comme le nombre d'iris bien classés sur le nombre total d'iris. Calculer la précision P sur la matrice de confusion précédente.

Solution

La précision P est donnée par le rapport du nombre d'iris bien classés sur le nombre total d'iris, soit

$$\boxed{P = \frac{46 + 48 + 45}{150} = \frac{139}{150} \approx 0.93.}$$

9) Implémenter une fonction `precision` qui prend comme argument une matrice de confusion M de taille 3×3 et qui renvoie la précision P. On précisera les spécifications de cette fonction et on la commentera. Exécuter cette fonction pas à pas sur la matrice de confusion de la Tab. I.1.

Solution

Voici une version rapide. Voir cours pour version plus longue (et plus simple).

```python
def precision(M):
    """
    Calcule la précision de l'algorithme des k plus proches voisins sur la matrice de confusion M.
    """
    return sum(M[i][i] for i in range(3))/sum(sum(M[i]) for i in range(3))
```

La fonction `precision` *renvoie la précision de l'algorithme des k plus proches voisins. Pour la matrice de confusion donnée, on a* `precision(M) = 0.93`.

10) On représente sur la Fig. I.2 le graphe de la précision $P(k)$ en fonction du nombre de plus proches voisins k. Commenter l'allure de cette courbe. Quelle valeur de k est optimale pour ce jeu de données ?

Solution

> *La précision P diminue avec le nombre de voisins k pour les valeurs de k suffisamment grandes, car le nombre de voisins devient trop important et l'algorithme perd en spécificité. La précision augmente avec k pour les valeurs de k suffisamment petites, car l'algorithme gagne en robustesse. La précision est maximale pour une valeur intermédiaire de k. La valeur optimale de k est celle qui maximise la précision, soit* $\boxed{k=10}$.

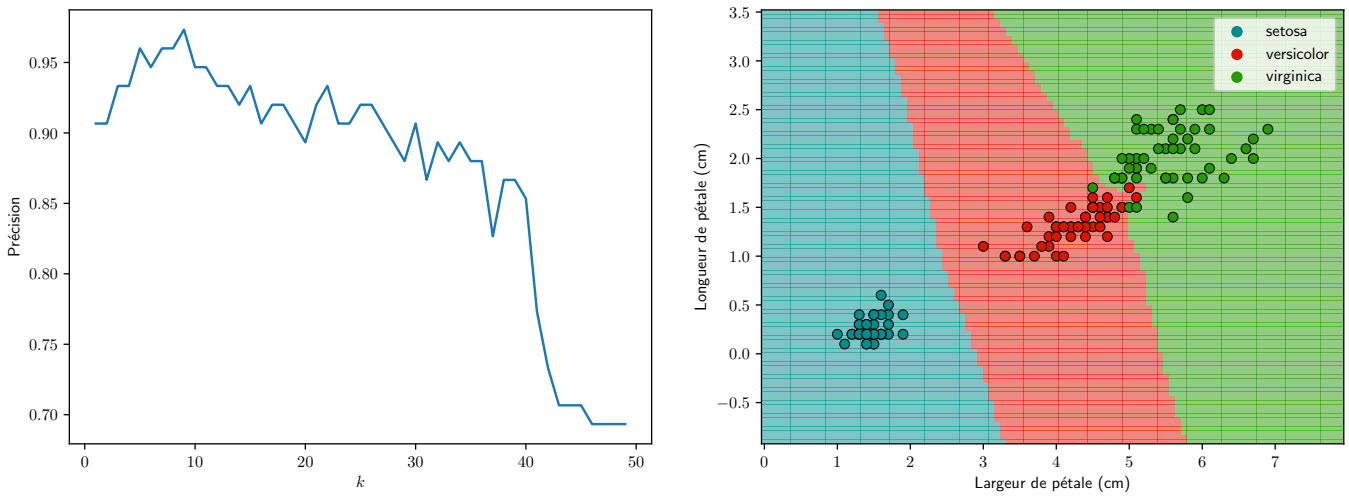
11) Pour sonder la qualité de l'algorithme des k plus proches voisins, on souhaite visualiser la frontière de décision.

La *frontière de décision* d'une classe i est définie comme la courbe $L_i(\ell)$ qui sépare deux régions du plan $\mathbb{R}^2$: la région R_i telle que pour tout point $y \in R_i$, la classe prédite est i, et la région $\overline{R_i}$ telle que pour tout point $y \in \overline{R_i}$, la classe prédite n'est pas i.

On visualise les frontières de décision pour $k = 3$ sur la Fig. I.2. Quelle classe serait prédite pour le point de coordonnées $(\ell = 3, L = 3)$?

Solution

> *Le point $(3, 3)$ est dans la région de décision de la classe versicolor. Il serait donc classé dans la variété* `'versicolor'`. *Sans tracer les frontières de décision, il serait moins évident de classer ce point à l'œil nu !*



(a) (b)

Fig. I.2 – (a) Précision P de l'algorithme des k plus proches voisins en fonction du nombre de voisins k. (b) Frontières de décision entres les trois régions bleu (setosa), rouge (versicolor) et verte (virginica) pour $k = 3$.

I.3 – Algorithme des k-moyennes

On suppose maintenant que les données de l'ensemble `iris` ne sont pas étiquetées, mais on sait que chaque fleur appartient à l'une des trois variétés d'iris : setosa, versicolor et virginica. On souhaite donc les classer par l'algorithme des k-moyennes avec $k = 3$. On rappelle le principe de l'algorithme des k-moyennes dans l'encadré ci-dessous.

(A) On choisit initialement k centres de classes c_1, ..., c_k.

(B) On associe à chaque donnée x la classe i telle que $d(x, c_i)$ est minimal.

(C) On évalue les barycentres b_1, ..., b_k des classes, que l'on choisit comme nouveaux centres.

Étape (A). Il existe plusieurs façons d'initialiser les centres : centres aléatoires dans l'espace des paramètres, centres aléatoires parmi les données, ou centres déterminés par une méthode heuristique. Un exemple de choix heuristique est le suivant : le premier centre c_1 est choisi aléatoirement parmi les données ; chaque centre suivant c_i est choisi comme la donnée la plus éloignée des centres c_1, ..., c_{i-1} déjà définis.

12) On définit ci-dessous une fonction `eloigne` qui renvoie la donnée x dont la somme des distances $d(x, c)$ aux centres c est maximale. Préciser les spécifications de cette fonction et expliquer la syntaxe de ce code.

```
    def eloigne(liste, E, d):
        return max(E, key=lambda x: sum(d(x,c) for c in liste))
```

Solution

La fonction `eloigne` renvoie la donnée x de l'ensemble E dont la somme des distances aux éléments de la liste est maximale. La fonction `max` renvoie l'élément de E qui maximise la fonction `lambda x: sum(d(x,c) for c in liste)`. La fonction `sum` renvoie la somme des distances de x aux centres de la liste. La méthode `lambda` permet de définir une fonction anonyme.

13) Écrire une fonction `init_centres` qui prend comme arguments un entier k, un ensemble de données `E`, une distance `d`, et qui renvoie une liste de k centres initialisés selon la méthode heuristique décrite ci-dessus. On précisera les spécifications de cette fonction et on la commentera. Quelles seraient approximativement les positions des centres c_2 et c_3 si le premier centre est choisi aléatoirement en $c_1 = (1.5, 0.5)$? Reproduire l'allure du graphe de la Fig. 1 en ajoutant de façon visible les points c_1, c_2 et c_3 pour illustrer votre réponse.

Solution

```
def init_centres(E,k,d):
    c = [sample(E,1)]
    for i in range(k-1):
        c.append(eloigne(c,E,d))
    return c
```

Les centres c_2 et c_3 seraient approximativement en $(4.5, 1.5)$ et $(5.5, 2.5)$.

14) Étape (B). On définit la fonction `minclasse` qui prend comme arguments une donnée `x`, une liste de centres `centres`, et une distance `d`, et qui renvoie le centre c le plus proche de `x`. Compléter la boucle `for` du code suivant pour obtenir cette fonction.

```
def minclasse(x, centres, d):
  """
  Entrée : x (tuple) : une donnée
           centres (list) : une liste de centres
           d (func) : une distance
  Renvoie le centre le plus proche de x parmi les centres.
  Sortie : c (tuple) : le centre le plus proche de x
  """
  c    = centres[0]
  dist = d(x,c)
  for y in centres[1:]:
      ...
  return c
```

Solution

```
def minclasse(x, centres, d):
    c    = centres[0]
    dist = d(x,c)
    for y in centres[1:]:
        if d(x,y) < dist:
            c    = y
            dist = d(x,y)
    return c
```

15) On définit ensuite la fonction `partition` de la façon suivante. Que réalise cette fonction ? Commenter le code et expliquer chaque étape.

```
def partition(E, centres, d):
    p = {}
```

```
    for x in E:
        c    = minclasse(x, centres, d)
        p[x] = centres.index(c)
    return p
```

Solution

16) Étape (C). Rappeler la définition du barycentre b d'un ensemble de points $x_i \in \mathbb{R}^2$, pour $i = 1, ..., n$. Implémenter une fonction `barycentre` qui prend comme argument une liste de points du plan, appelée `points`, et qui renvoie le barycentre de ces points. On précisera les spécifications de cette fonction et on la commentera.

On souhaite maintenant appliquer l'algorithme des k-moyennes à l'ensemble de données `iris_inconnues` pour $k = 3$. On initialise les centres de classes par la méthode heuristique décrite ci-dessus.

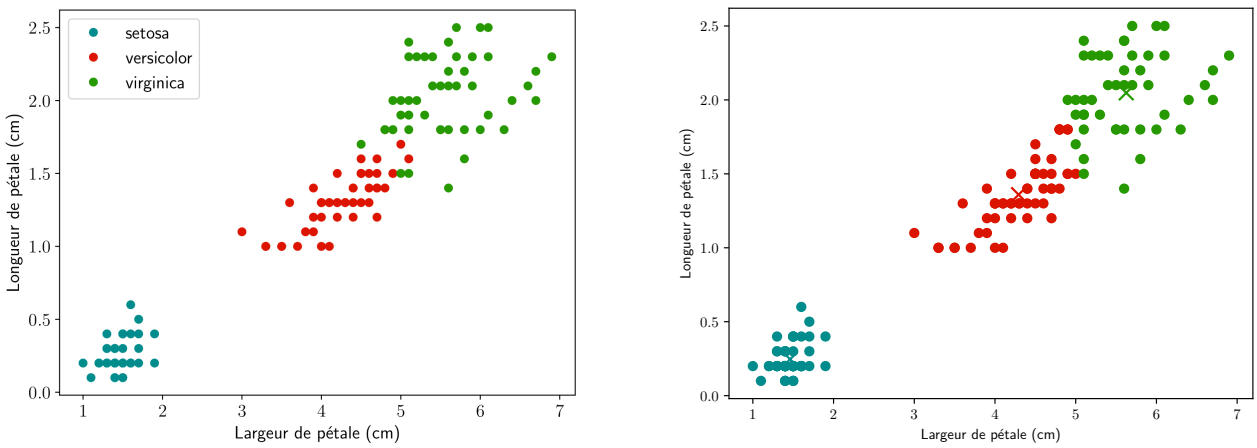
17) Définir une fonction commentée `kmoy` qui prend comme arguments un entier k, un ensemble de données `E`, une distance `d`, et un nombre d'itérations n, et qui renvoie la classification des données de `E` obtenue par n itérations de l'algorithme des k-moyennes. On pourra utiliser les fonctions `init_centres`, `partition`, et `barycentre` définies précédemment. On pourra s'aider du code suivant.

```
def kmoy(k, E, d, n):
    """
    Entrée : k (int) : nombre de classes
             E (list) : ensemble de données
             d (func) : une distance
             n (int) : nombre d'itérations
    Renvoie la classification des données E obtenue par l'algorithme des k-moyennes.
    Sortie : p (dict) : la classification des données E
    """
    ...
    for j in range(n):
      ...
    return p
```

Solution

```
def kmoy(k, E, d, n):
    """
    Entrée : k (int) : nombre de classes
             E (list) : ensemble de données
             d (func) : une distance
             n (int) : nombre d'itérations
    Renvoie la classification des données E obtenue par l'algorithme des k-moyennes.
    Sortie : p (dict) : la classification des données E
    """
    centres = init_centres(k, E)
    for j in range(n):
        p = partition(E, centres, d)
        i = 0
        for classe in p.values():
            centres[i] = barycentre([x for x in p if p[x] == classe])
            i += 1
    return p
```

18) On rappelle ci-dessous à gauche la classification connue des variétés d'iris, que l'on a reporté de la Fig. 1. On donne ci-dessous à droite le classement obtenu par l'algorithme des k-moyennes pour $k = 3$ sur la Fig. ??. Commenter les résultats obtenus.



(a) (b)

Fig. I.3 – (a) Classification connue des variétés d'iris. (b) Classement des variétés d'iris prédit par l'algorithme des k-moyennes pour $k = 3$. Le nombre d'itérations est choisi de sorte que les centres soient stationnaires, c'est-à-dire n'évoluent plus entre deux itérations. Les centres finaux sont représentés par des croix.

19) Pensez-vous que l'algorithme des k-moyennes est adapté pour classer un ensemble de données réparties comme sur la Fig. I.4 ci-dessous ? Justifier.

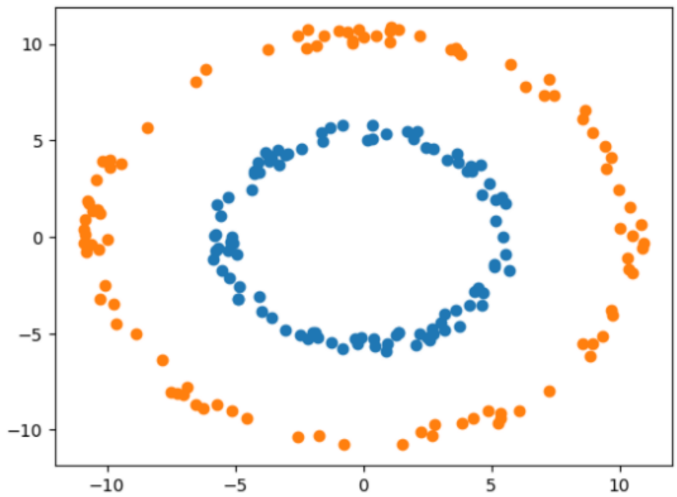


Fig. I.4 – Exemple de données non linéairement séparables.

I.4 – Méthode du coude

Nous avons imposé le nombre de classes à $k = 3$ sachant que les données correspondent à l'une ou l'autre des trois variétés d'iris : setosa, versicolor et virginica. En présence de données brutes issues du terrain, et en particulier de plantes mal connues, le nombre de variétés de fleur n'est pas connu. On peut alors appliquer l'algorithme des k-moyennes pour différents nombres de classes k et comparer les résultats. On définit les notions suivantes.

La *variance* d'un ensemble de données X de barycentre b est la grandeur

$$V(X) = \sum_{x \in X} d(x, b)^2.$$

L'*inertie* d'une classification $p = (X_i)_{i=1,\dots,k}$ d'un ensemble de données E en k classes est la somme des variances des classes, soit

$$I(p) = \sum_{i=1}^{k} V(X_i).$$

20) *Exemple.* Calculer les barycentres b_1 et b_2 et les variances V_1 et V_2 des classes X_1 et X_2 suivantes : $X_1 = \{(1,1),(3,3)\}$, $X_2 = \{(5,4),(7,4)\}$. Calculer l'inertie I de la classification $p = (X_1, X_2)$. Représenter ces données sur un graphe.

21) Justifier que minimiser l'inertie $I(p)$ est un bon critère pour juger de la qualité d'un classement de données par une classification p. Comment évolue l'inertie $I(p)$ au cours des différentes itérations de l'algorithme des k-moyennes ? Commenter.

22) On représente ci-dessous l'inertie de la classification obtenue par l'algorithme des k-moyennes en fonction du nombre de classes k. Pour déterminer la valeur optimale de k, il est d'usage de recourir à la *méthode du coude* : on choisit la valeur de k pour laquelle l'inertie diminue significativement avant cette valeur mais ne varie presque plus après cette valeur. Justifier ce critère. Quelle valeur de k est optimale pour le classement des données d'iris d'après le graphe de la Fig. I.5 ci-dessous ?

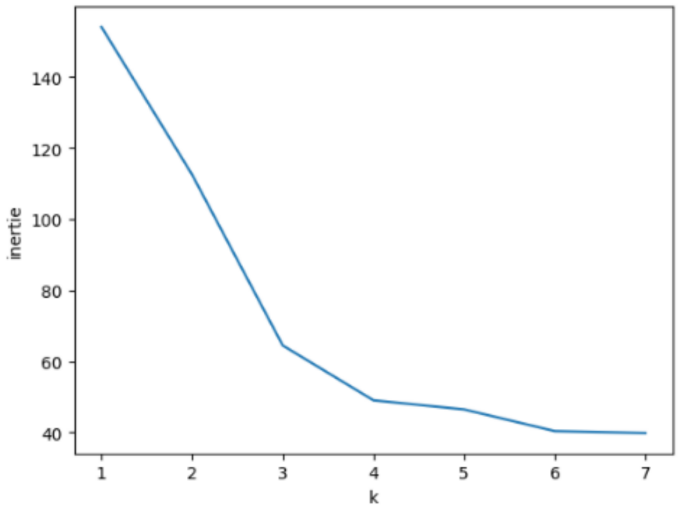


Fig. I.5 – Inertie $I(p)$ de la classification p obtenue par l'algorithme des k-moyennes sur les données d'iris pour différents nombres de classes k.